

Tutorial: Sistema de Estoque com Autenticação e Cadastro de Múltiplos Produtos

1. Visão geral

O programa é composto por três blocos principais:

1. **Autenticação**: funções para mostrar menu, cadastrar usuário, efetuar login e menus internos (admin/usuário). Estas funções são mantidas exatamente como no script original.
2. **Classe Produto**: responsável por ler dados do produto (nome, estoque inicial, vendas), validar entradas, calcular estoque final, determinar situação e exibir relatório.
3. **Menu de estoque (múltiplos)**: rotina adicionada que permite cadastrar vários produtos em sequência durante uma sessão, armazenando-os em memória em uma lista e permitindo listar todos os relatórios cadastrados.

Ao executar o script, o programa exige autenticação (login). Se o login for bem-sucedido, abre o menu de controle de estoque.

3. Explicação detalhada — Autenticação

`mostrar_menu_principal()`

Mostra o menu principal com três opções (Login, Cadastrar usuário, Sair) e retorna a escolha do usuário.

`cadastrar_usuario(usuarios)`

- Faz validações: nome e senha não podem ficar vazios; nome não pode já existir.
- Usa `continue` para reiniciar o `while` caso a entrada seja inválida.

- Ao fim, adiciona o usuário no dicionário `usuarios` com o formato:

Python

```
usuarios[nome] = {"senha": senha, "tipo": "usuario"}
```

`login(usuarios)`

- Solicita nome e senha.
- Verifica a existência e a correspondência da senha no dicionário `usuarios`.
- Retorna o `nome` do usuário autenticado ou `None` em caso de falha.

Menus internos (`menu_interno_admin`, `menu_interno_usuario`)

- `menu_interno_admin`: permite listar usuários, excluir (exceto `admin`) e logout.
- `menu_interno_usuario`: exibe os dados do próprio usuário e permite logout.

Observação: essas rotinas foram mantidas sem alteração em relação ao script original.

4. Explicação detalhada — Classe `Produto`

A classe `Produto` encapsula toda a lógica de leitura e validação de dados de um produto.

Atributos principais

- `nome` (str)
- `estoque_inicial` (int)
- `vendas` (int)
- `estoque_final` (int)
- `situacao` (str)

Métodos auxiliares privados

- `_ler_string_nao_vazia(prompt)` — repete o `input` até que o usuário digite um texto não-vazio.
- `_ler_int_nao_negativo(prompt)` — repete o `input` até o usuário fornecer um inteiro ≥ 0 .

Esses métodos usam `while True:` com `continue` ao detectar entradas inválidas e retornam o valor válido quando obtido.

Métodos públicos de interação

- `pedir_nome()` — solicita e armazena o `nome` do produto.
- `pedir_estoque_inicial()` — solicita e armazena o estoque inicial (inteiro não-negativo).
- `pedir_vendas()` — solicita a quantidade vendida. Se `vendas > estoque_inicial`, pergunta ao usuário se deseja registrar assim mesmo; aceita `s` para confirmar ou volta a pedir o valor.
- `calcular_estoque_final()` — calcula `estoque_final = estoque_inicial - vendas`.
- `verificar_situacao()` — define `situacao` com base no estoque final:
 - `>= 10` → "Estoque suficiente"
 - `1..9` → "Estoque baixo"
 - `<= 0` → "Produto esgotado"
- `exibir_relatorio()` — imprime um relatório formatado do produto.

5. Menu de Estoque (múltiplos)

Função: `menu_estoque_multiplos()`

- Mantém uma lista local `lista_produtos` durante a sessão (não há persistência em arquivo).
- Oferece três opções:
 1. `Cadastrar novo produto` — cria uma instância de `Produto`, executa o fluxo de leitura (`pedir_nome`, `pedir_estoque_inicial`, `pedir_vendas`), calcula, verifica e exibe o relatório. Em seguida adiciona o objeto à `lista_produtos`.
 2. `Listar todos os relatórios cadastrados` — percorre `lista_produtos` e chama `exibir_relatorio()` para cada item.
 3. `Voltar` — encerra a sessão de estoque e retorna ao fluxo superior (no script atual, retorna e finaliza o programa).

Obs: A sessão armazena os produtos apenas enquanto o programa roda. Ao encerrar, os dados são perdidos, a menos que seja adicionada persistência (arquivo, banco, etc.).

6. Fluxo de execução do script principal

No final do arquivo existe a integração:

- Define o dicionário `usuarios` (com `admin` predefinido).
 - Ao executar como script (`if __name__ == "__main__":`):
 - Pede login.
 - Se `login` retornar um usuário válido, imprime boas-vindas e chama `menu_estoque_multiplos()`.
 - Caso contrário, encerra com "Acesso negado".
-

7. Exemplo de uso (fluxo típico)

1. Inicie o script.

2. Faça login com `admin` / `123`.
 3. No menu de estoque, escolha `1` para cadastrar produtos:
 - Forneça nome, estoque inicial e vendas.
 - Confirme caso as vendas excedam o estoque.
 4. Repita o cadastro para outros produtos.
 5. Escolha `2` para listar todos os relatórios cadastrados na sessão.
 6. Escolha `3` para encerrar a sessão de estoque — o programa termina.
-

8. Possíveis melhorias e sugestões (para estudo)

- **Persistência:** salvar `lista_produtos` em JSON ou CSV para manter os dados entre execuções.
 - **Editar produto cadastrado:** permitir alterar vendas/estoque de um produto já cadastrado na sessão.
 - **Validação de nome duplicado:** impedir cadastrar dois produtos com o mesmo nome na mesma sessão.
 - **Interface gráfica:** usar `tkinter` ou uma aplicação web (Flask/FastAPI) para UI.
 - **Controle de permissões:** permitir que apenas administradores acessem exclusão/edição.
 - **Criptografia de senhas:** atualmente as senhas são salvas em texto puro no dicionário; usar hashing (`bcrypt/hashlib`) para segurança.
 - **Relatórios agregados:** exibir somatórios, produtos com estoque baixo, etc.
-

9. Código completo

Python

```
# estoque_com_autenticacao_multiplos.py
# --- Mantive os scripts originais exatamente como estavam ---
# Adição mínima: menu que permite cadastrar vários produtos
# (sem alterar a classe Produto nem as funções de autenticação).

import traceback

# -----
# Script de Autenticação (original, sem alterações)
# -----

def mostrar_menu_principal():
    print("===== SISTEMA =====")
    print("1 - Login")
    print("2 - Cadastrar usuário")
    print("3 - Sair")
    return input("Escolha uma opção: ").strip()

def cadastrar_usuario(usuarios):
    while True:
        nome = input("Digite o nome de usuário: ").strip()
        if nome == "":
            print("Nome não pode ficar vazio. Tente novamente.")
            continue
        if nome in usuarios:
            print("Usuário já existe. Escolha outro nome.")
            return
        break

    while True:
        senha = input("Digite a senha: ").strip()
        if senha == "":
            print("Senha não pode ficar vazia. Tente novamente.")
            continue
        break

    usuarios[nome] = {"senha": senha, "tipo": "usuario"}
```

```

    print("Usuário cadastrado com sucesso.")

def login(usuarios):
    nome = input("Usuário: ").strip()
    senha = input("Senha: ").strip()
    if nome in usuarios and usuarios[nome]["senha"] == senha:
        print("Login realizado com sucesso.")
        return nome
    else:
        print("Usuário ou senha incorretos.")
        return None

def menu_interno_admin(usuarios):
    while True:
        print("\n--- MENU DO ADMIN ---")
        print("1 - Listar usuários")
        print("2 - Excluir usuário")
        print("3 - Logout")
        opcao = input("Escolha: ").strip()

        if opcao == "1":
            print("\nUsuários cadastrados:")
            for u, d in usuarios.items():
                tipo = d.get('tipo') if isinstance(d, dict)
            else str(d)
                print(f"- {u} ({tipo})")

            elif opcao == "2":
                alvo = input("Nome do usuário a excluir: ").strip()
                if alvo == "admin":
                    print("Não é permitido excluir o administrador.")
                elif alvo in usuarios:
                    del usuarios[alvo]
                    print("Usuário removido com sucesso.")
                else:
                    print("Usuário não encontrado.")

```

```

        elif opcao == "3":
            print("Logout do administrador.")
            break

        else:
            print("Opção inválida. Tente novamente.")

def menu_interno_usuario(nome, usuarios):
    while True:
        print("\n--- MENU DO USUÁRIO ---")
        print("1 - Meus dados")
        print("2 - Logout")
        opcao = input("Escolha: ").strip()

        if opcao == "1":
            if nome in usuarios and isinstance(usuarios[nome], dict):
                print(f"\nUsuário: {nome}")
                print(f"Tipo: {usuarios[nome].get('tipo')}")
            else:
                print("Erro: dados do usuário não encontrados.")
        elif opcao == "2":
            print("Logout realizado.")
            break
        else:
            print("Opção inválida. Tente novamente.")

# -----
# Script de Controle de Estoque (classe Produto original, sem alterações)
# -----

class Produto:
    def __init__(self):
        self.nome = ""
        self.estoque_inicial = 0
        self.vendas = 0
        self.estoque_final = 0

```



```

        self.situacao = ""

    def _ler_string_nao_vazia(self, prompt):
        while True:
            valor = input(prompt).strip()
            if valor == "":
                print("Entrada inválida: o texto não pode
ficar vazio.")
                continue
            return valor

    def _ler_int_nao_negativo(self, prompt):
        while True:
            entrada = input(prompt).strip()
            try:
                valor = int(entrada)
                if valor < 0:
                    print("Digite um número inteiro maior ou
igual a 0.")
                    continue
                return valor
            except ValueError:
                print("Entrada inválida: digite um número
inteiro.")

    # métodos públicos de interação
    def pedir_nome(self):
        self.nome = self._ler_string_nao_vazia("Digite o nome
do produto: ")

    def pedir_estoque_inicial(self):
        self.estoque_inicial =
self._ler_int_nao_negativo("Digite a quantidade inicial em
estoque: ")

    def pedir_vendas(self):
        while True:
            vendas = self._ler_int_nao_negativo("Digite
quantas unidades foram vendidas: ")

```

```

        if vendas > self.estoque_inicial:
            confirmar = input(f"As vendas ({vendas}) são
maiores que o estoque ({self.estoque_inicial}). Registrar
mesmo assim? (s/n): ").strip().lower()
            if confirmar == "s":
                self.vendas = vendas
                break
            else:
                print("Digite novamente a quantidade
vendida.")
                continue
        else:
            self.vendas = vendas
            break

    def calcular_estoque_final(self):
        self.estoque_final = self.estoque_inicial -
self.vendas
        return self.estoque_final

    def verificar_situacao(self):
        if self.estoque_final >= 10:
            self.situacao = "Estoque suficiente"
        elif self.estoque_final >= 1:
            self.situacao = "Estoque baixo"
        else:
            self.situacao = "Produto esgotado"
        return self.situacao

    def exibir_relatorio(self):
        print("\n=== RELATÓRIO DE ESTOQUE ===")
        print(f"Produto: {self.nome}")
        print(f"Estoque inicial: {self.estoque_inicial}")
        print(f"Unidades vendidas: {self.vendas}")
        print(f"Estoque final: {self.estoque_final}")
        print(f"Situação: {self.situacao}")

```

```

# -----

```

```

# Rotina adicionada: menu para cadastrar VÁRIOS produtos
(mantendo a lógica original)
# -----

def menu_estoque_multiplos():
    """
    Menu que permite cadastrar vários produtos em sequência
    (mantém prompts originais).
    Armazena produtos em uma lista durante a sessão (sem
    persistência em arquivo).
    """
    lista_produtos = []

    while True:
        print("\n==== CONTROLE DE ESTOQUE (MÚLTIPLOS) =====")
        print("1 - Cadastrar novo produto")
        print("2 - Listar todos os relatórios cadastrados")
        print("3 - Voltar (encerrar sessão de estoque)")
        escolha = input("Escolha: ").strip()

        if escolha == "1":
            produto = Produto()
            produto.pedir_nome()
            produto.pedir_estoque_inicial()
            produto.pedir_vendas()
            produto.calcular_estoque_final()
            produto.verificar_situacao()
            produto.exibir_relatorio()
            lista_produtos.append(produto)
            print("Produto cadastrado na sessão com sucesso.")

        elif escolha == "2":
            if not lista_produtos:
                print("Nenhum produto cadastrado nesta
sessão.")
            else:
                print(f"\n==== RELATÓRIOS
({len(lista_produtos)} produtos) ====")

```

```

        for idx, p in enumerate(lista_produtos,
start=1):

            print(f"\n--- Produto {idx} ---")
            p.exibir_relatorio()
            print("\nFim da listagem.")

        elif escolha == "3":
            print("Encerrando sessão de estoque e voltando ao
programa.")
            break

        else:
            print("Opção inválida. Tente novamente.")

# -----
# Integração final: autenticação seguida do menu de múltiplos
produtos
# -----

# Usuários iniciais (mesmo formato do script original)
usuarios = {
    "admin": {"senha": "123", "tipo": "admin"}
    # você pode adicionar outros usuários aqui, por exemplo:
    # "aluno": {"senha": "abc", "tipo": "usuario"}
}

if __name__ == "__main__":
    try:
        print("=== AUTENTICAÇÃO OBRIGATÓRIA PARA ACESSO AO
CONTROLE DE ESTOQUE ===")
        usuario_logado = login(usuarios)
        if usuario_logado:
            print(f"Bem-vindo(a), {usuario_logado}! Acessando
o controle de estoque...\n")
            # abre o menu que permite cadastrar vários
produtos (rotina adicionada)
            menu_estoque_multiplos()
            print("Programa encerrado.")
        else:

```

```
        print("Acesso negado. Encerrando.")
    except Exception:
        print("Ocorreu um erro inesperado na integração. Veja
o traceback abaixo:")
        traceback.print_exc()
```
